



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

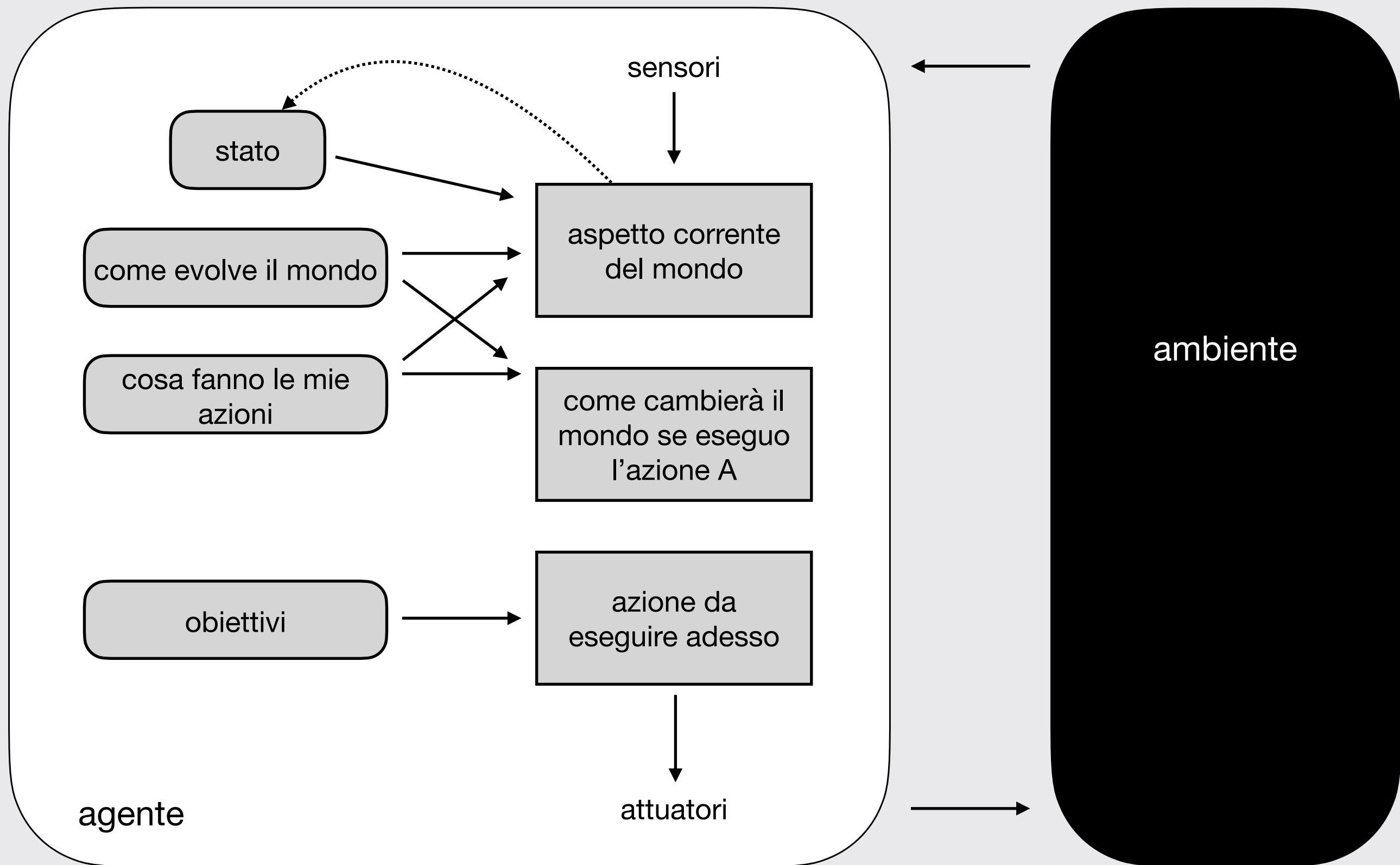
Laurea triennale in Informatica

Fondamenti di Intelligenza Artificiale

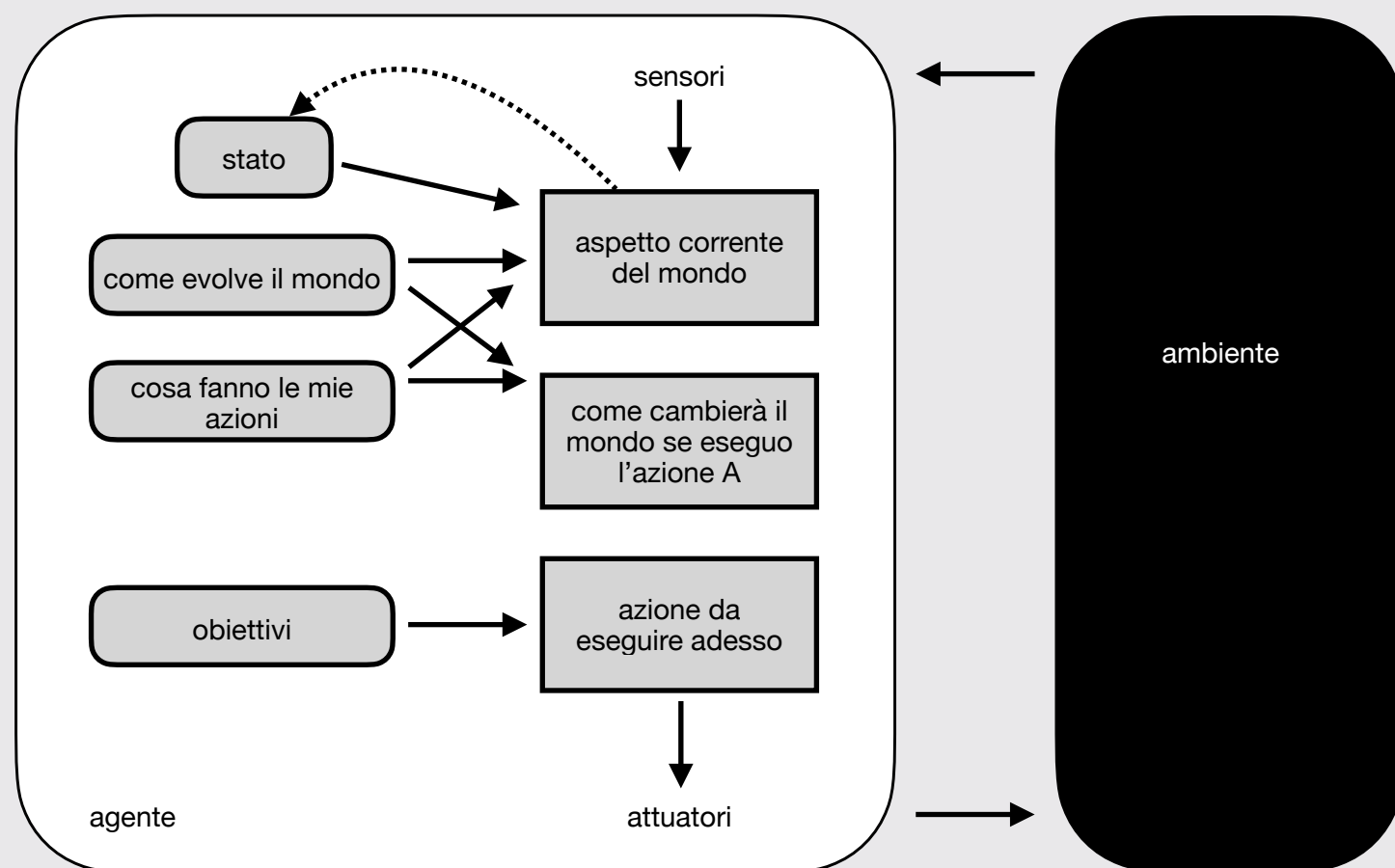
Lezione 3 - Formulazione di problemi di ricerca



Agenti Basati su Obiettivi



Agenti Basati su Obiettivi



Flessibilità

La conoscenza è rappresentata e può essere modificata, pertanto è da considerare come un agente più **flessibile** degli altri.

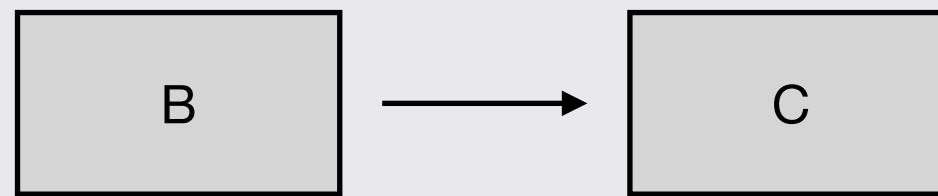
In un agente reattivo, dovremmo riscrivere molte delle regole condizione-azione; qui, invece, il comportamento può essere adattato.

Filosofia degli agenti basati su obiettivi

Talvolta, scegliere un'azione in base ad un obiettivo è molto semplice, quando questo può essere raggiunto in un solo passo. Altre volte è più difficile, ad esempio quando l'agente deve considerare lunghe sequenze di azioni per trovare il "cammino" che porta al risultato desiderato.

Il funzionamento di tali agenti **non** prevede l'uso di regole condizione-azione (tipiche degli agenti basati su modello), poiché considera il futuro in relazione a due domande: "Cosa accadrà se compio l'azione A?" e "Se faccio A, sarò soddisfatto?"

Agenti Basati su Obiettivi - Funzionamento



La rappresentazione atomica

In una rappresentazione atomica ogni stato del mondo è indivisibile, ovvero non ha una struttura interna. L'unica proprietà discernibile di uno stato è il fatto di essere uguale o diversa da un altro stato -> non conosciamo la struttura interna di uno stato.

Gli algoritmi alla base di ricerca e giochi lavorano tutti con rappresentazioni atomiche o, quantomeno, considerano le rappresentazioni come se fossero atomiche.

Risoluzione di problemi con la ricerca

Nel contesto di questa parte del corso, ci focalizzeremo su un particolare tipo di agenti basati su obiettivi, chiamati **agenti risolutori di problemi**.

Tali agenti si pongono uno o più obiettivi da raggiungere ed eseguono azioni che massimizzano le chance del loro raggiungimento. Utilizzano, pertanto, rappresentazioni atomiche del mondo, i cui stati verranno usati per via via trovare una soluzione accettabile al problema trattato.

Agenti Risolutori di Problemi

Cominciamo con un esempio

Supponiamo che un agente X sia in vacanza ad Arad, in Romania. La sua **misura di prestazione** sarà composta da diversi fattori: vorrà farsi una bella tintarella (ad Arad?), migliorare il suo rumeno, ammirare i panorami più belli, passare notti divertenti, evitare i postumi di una sbronza, e così via.

In un contesto del genere, l'obiettivo dell'agente risulta essere complesso in quanto dovrà trovare una serie di compromessi che bilancino i vari obiettivi.

Dovrà necessariamente evitare eccessi come, ad esempio:

1. Tornare a casa alle 7 di mattina completamente ubriaco;
2. Passare troppo tempo a Palazzo Cenad nel tentativo di apprezzare la combinazione di stili, eclettico e neoclassico e dalla ricca architettura ornamentale, in stile classico e rinascimentale.



Agenti Risolutori di Problemi

Cominciamo con un esempio

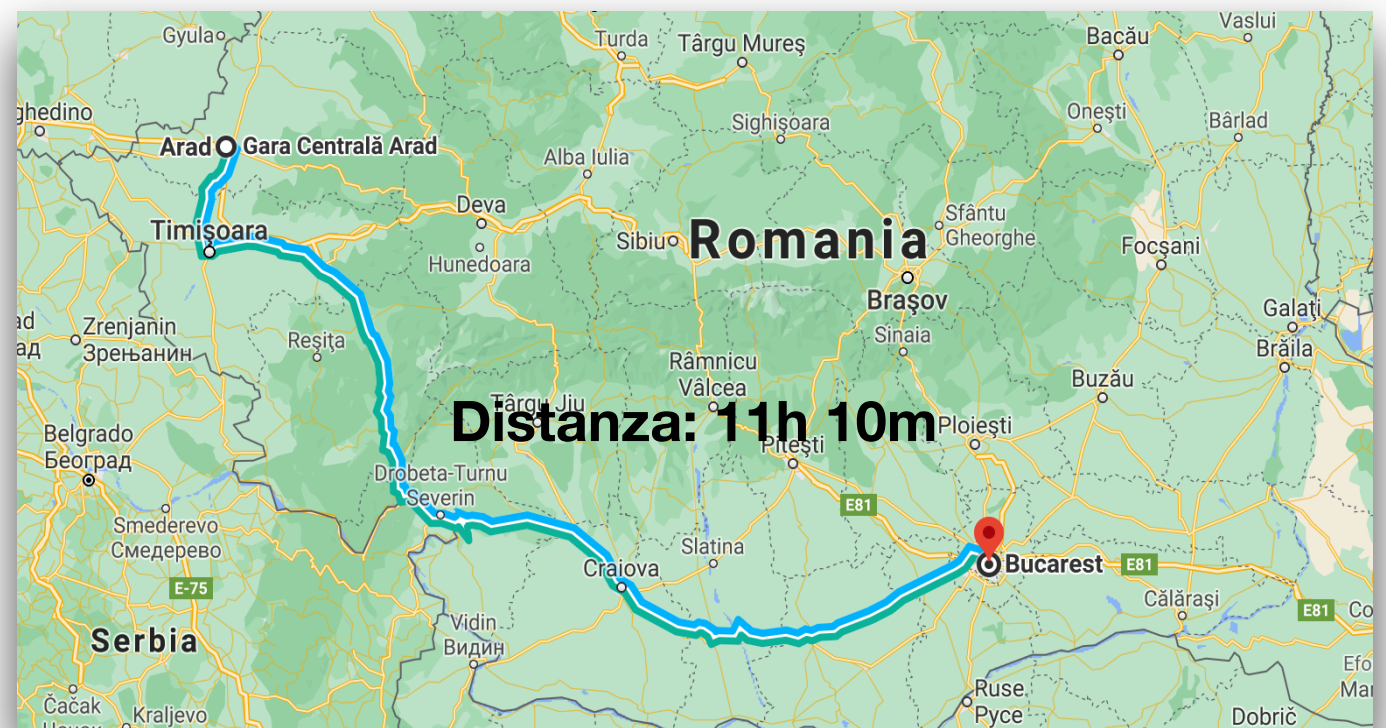
Supponiamo che un agente X sia in vacanza ad Arad, in Romania. La sua **misura di prestazione** sarà composta da diversi fattori: vorrà farsi una bella tintarella (ad Arad?), migliorare il suo rumeno, ammirare i panorami più belli, passare notti divertenti, evitare i postumi di una sbronza, e così via.

Supponiamo adesso che l'agente abbia un biglietto aereo non rimborsabile per la partenza da Bucarest il giorno successivo.

Dato questo presupposto, sembra sensato che l'agente adotti l'**obiettivo** di arrivare a Bucarest in tempo utile per potersi imbarcare.

Tutte le azioni che non consentiranno all'agente di raggiungere Bucarest in tempo possono pertanto **essere scartate**.

In termini più pratici, questo implica che il cosiddetto **spazio delle soluzioni** potrà essere sostanzialmente ridotto, velocizzando il raggiungimento di una **soluzione**.



Agenti Risolutori di Problemi

Cominciamo con un esempio

Supponiamo che un agente X sia in vacanza ad Arad, in Romania. La sua **misura di prestazione** sarà composta da diversi fattori: vorrà farsi una bella tintarella (ad Arad?), migliorare il suo rumeno, ammirare i panorami più belli, passare notti divertenti, evitare i postumi di una sbronza, e così via.

Supponiamo adesso che l'agente abbia un biglietto aereo non rimborsabile per la partenza da Bucarest il giorno successivo.

Dato questo presupposto, sembra sensato che l'agente adotti l'**obiettivo** di arrivare a Bucarest in tempo utile per potersi imbarcare.

Formulazione dell'obiettivo: La formulazione dell'obiettivo, basata sulla situazione corrente e sulla misura di prestazione dell'agente, rappresenta necessariamente *il primo passo nella risoluzione dei problemi*.

Obiettivo: Questo è rappresentato da un insieme ammissibile di stati del mondo. Più in dettaglio, è rappresentato da tutti e soli quegli stati in cui l'obiettivo è soddisfatto.

Il compito dell'agente è determinare come agire, ora e nel futuro, per raggiungere uno stato obiettivo. Per fare ciò, l'agente deve decidere quali tipi di azioni e stati prendere in considerazione. Approfondiamo questo aspetto...

Agenti Risolutori di Problemi

Cominciamo con un esempio

Supponiamo che un agente X sia in vacanza ad Arad, in Romania. La sua **misura di prestazione** sarà composta da diversi fattori: vorrà farsi una bella tintarella (ad Arad?), migliorare il suo rumeno, ammirare i panorami più belli, passare notti divertenti, evitare i postumi di una sbronza, e così via.

Supponiamo adesso che l'agente abbia un biglietto aereo non rimborsabile per la partenza da Bucarest il giorno successivo.

Dato questo presupposto, sembra sensato che l'agente adotti l'**obiettivo** di arrivare a Bucarest in tempo utile per potersi imbarcare.

Se l'agente cercasse di considerare azioni di “basso livello”, quali:

- “Muovi il piede sinistro di un centimetro”;
- “Gira il volante a sinistra di un grado”;

Allora con ogni probabilità non riuscirebbe neanche ad uscire dal parcheggio! A quel grado di dettaglio c'è **troppa incertezza** e una soluzione sarebbe composta da un **numero eccessivo di passi**. Pertanto, l'agente dovrà scegliere una **giusta granularità**.

Formulazione del problema: La formulazione del problema è il processo che porta a decidere, dato un obiettivo, quali azioni e quali stati considerare.

Agenti Risolutori di Problemi

Cominciamo con un esempio

Supponiamo che un agente X sia in vacanza ad Arad, in Romania. La sua **misura di prestazione** sarà composta da diversi fattori: vorrà farsi una bella tintarella (ad Arad?), migliorare il suo rumeno, ammirare i panorami più belli, passare notti divertenti, evitare i postumi di una sbronza, e così via.

Supponiamo adesso che l'agente abbia un biglietto aereo non rimborsabile per la partenza da Bucarest il giorno successivo.

Dato questo presupposto, sembra sensato che l'agente adotti l'**obiettivo** di arrivare a Bucarest in tempo utile per potersi imbarcare.

Supponiamo, quindi, che l'agente esaminerà azioni **al livello di guidare da una grande città ad un'altra**. Ogni stato corrisponderà a trovarsi in una particolare città.

Ci sono tre strade che portano fuori città: una in direzione Sibiu, una verso Timisoara, la terza verso Zerind.

Chiaramente, nessuna opzione soddisfa l'obiettivo; pertanto, l'agente non saprà dove andare. In altri termini, in questo stato l'agente **non possiede abbastanza informazioni sullo stato risultante da una azione**.



Agenti Risolutori di Problemi

Cominciamo con un esempio

Supponiamo che un agente X sia in vacanza ad Arad, in Romania. La sua **misura di prestazione** sarà composta da diversi fattori: vorrà farsi una bella tintarella (ad Arad?), migliorare il suo rumeno, ammirare i panorami più belli, passare notti divertenti, evitare i postumi di una sbronza, e così via.

Supponiamo adesso che l'agente abbia un biglietto aereo non rimborsabile per la partenza da Bucarest il giorno successivo.

Dato questo presupposto, sembra sensato che l'agente adotti l'**obiettivo** di arrivare a Bucarest in tempo utile per potersi imbarcare.

Supponiamo, quindi, che l'agente esaminerà azioni **al livello di guidare da una grande città ad un'altra**. Ogni stato corrisponderà a trovarsi in una particolare città.

Se l'ambiente fosse **ignoto**, l'agente dovrebbe scegliere un'azione a caso. Se supponessimo invece che l'agente abbia una mappa della Romania, allora potrebbe collezionare informazioni sull'ambiente circostante. Una volta ottenute tali informazioni, l'agente può usarle per considerare i passi *successivi*.

Un agente che ha a disposizione diverse opzioni immediate di **valore sconosciuto** può decidere cosa fare *esaminando le azioni future* che alla fine porteranno a stati di **valore conosciuto**.

Agenti Risolutori di Problemi

Esaminare le azioni future: che significa?

Ipotizziamo che l'ambiente sia **osservabile**, pertanto l'agente conosce *sempre* lo stato corrente del mondo. Nell'esempio precedente, questo significherebbe che ogni città abbia un cartello che indichi la sua presenza ai guidatori in arrivo.

Ipotizziamo che l'ambiente sia **discreto**, pertanto in qualsiasi stato esiste un numero finito di azioni tra cui scegliere.

Ipotizziamo che l'ambiente sia **noto**, pertanto l'agente saprà sempre quali stati saranno raggiunti da ciascuna azione. Nei problemi di navigazione, questo requisito è soddisfatto a patto che si abbia a disposizione una mappa accurata.

Ipotizziamo che l'ambiente sia **deterministico**, pertanto ogni azione avrà uno ed un solo risultato.

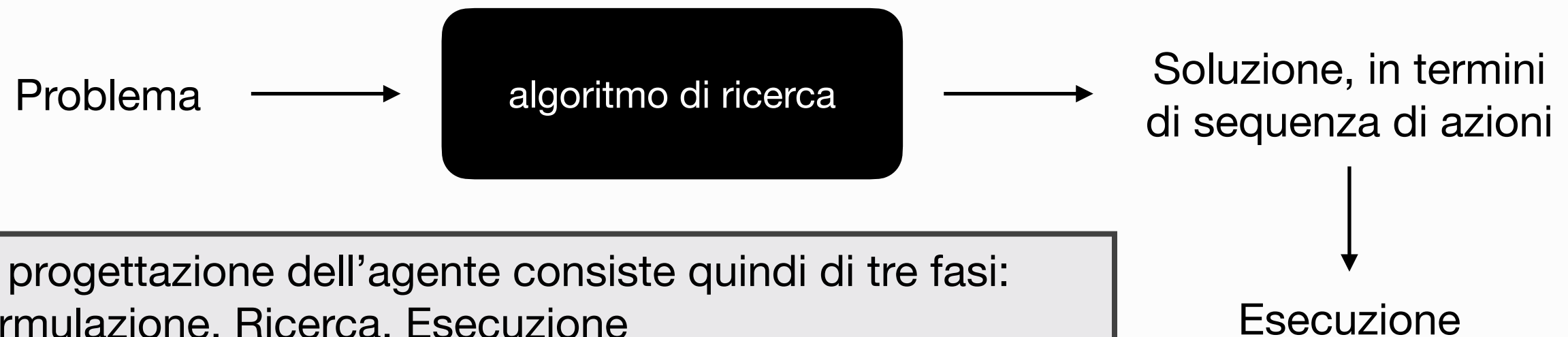
Sotto queste ipotesi, la soluzione di qualsiasi problema è rappresentata da una **sequenza fissata di azioni**. In altri termini, una soluzione potrebbe essere implementata come una strategia ramificata che raccomandi azioni future diverse in base alle percezioni giunte.

In una situazione del genere, l'agente in esempio saprebbe esattamente dove si troverà dopo la prima azione e che cosa percepirà, e così via. Così facendo, potrebbe identificare una soluzione specificata da una sequenza di azioni che porti da una città all'altra fino a Bucarest.

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



La progettazione dell'agente consiste quindi di tre fasi: Formulazione, Ricerca, Esecuzione

```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
               stato, una descrizione dello stato corrente del mondo;
               goal, un obiettivo, inizialmente null;
               problema, una formulazione di problema;

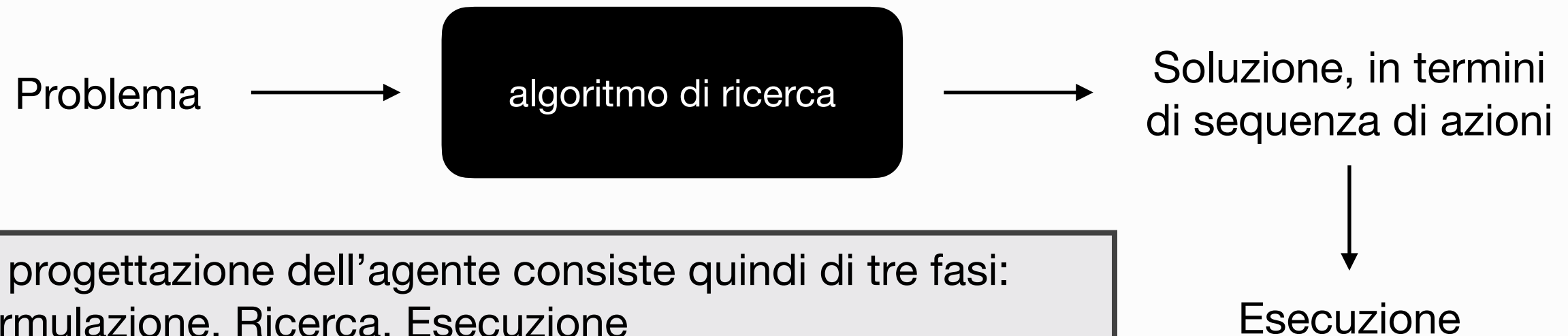
  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla

  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



La progettazione dell'agente consiste quindi di tre fasi: Formulazione, Ricerca, Esecuzione

```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
               stato, una descrizione dello stato corrente del mondo;
               goal, un obiettivo, inizialmente null;
               problema, una formulazione di problema;

  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla

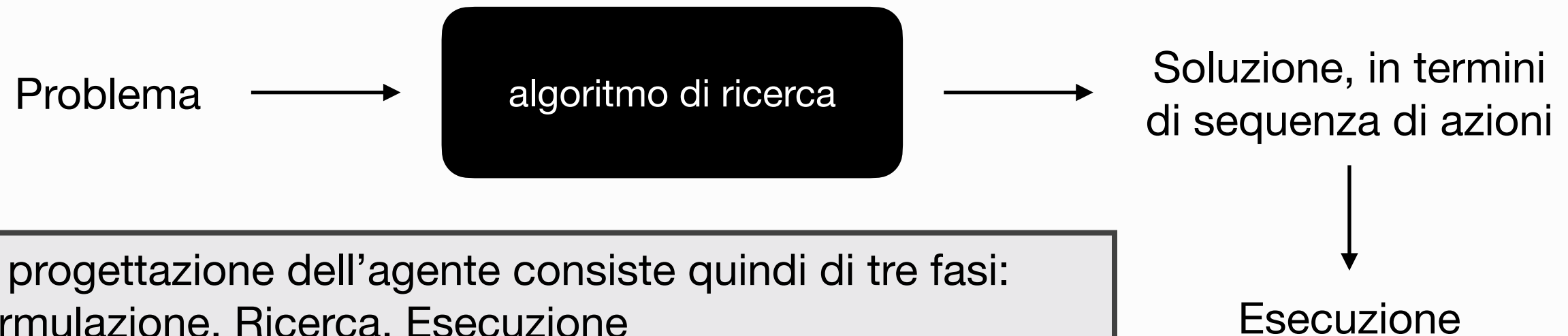
  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

←
/* La funzione prende in input la
percezione corrente dell'agente */

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



La progettazione dell'agente consiste quindi di tre fasi: Formulazione, Ricerca, Esecuzione

```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
               stato, una descrizione dello stato corrente del mondo;
               goal, un obiettivo, inizialmente null;
               problema, una formulazione di problema;

  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla

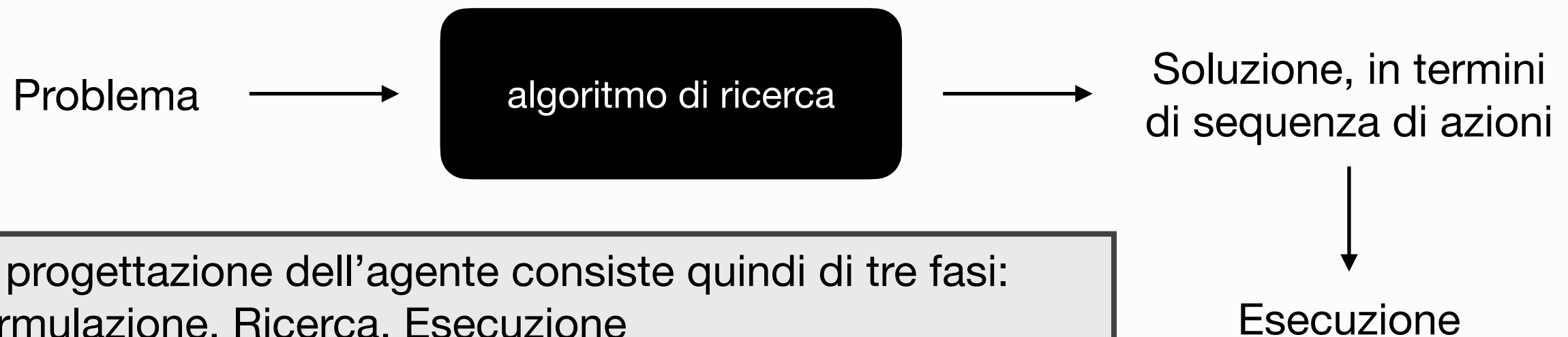
  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

/* Nella prima fase, le variabili necessarie alla risoluzione del problema sono istanziate */

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



La progettazione dell'agente consiste quindi di tre fasi: Formulazione, Ricerca, Esecuzione

```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
               stato, una descrizione dello stato corrente del mondo;
               goal, un obiettivo, inizialmente null;
               problema, una formulazione di problema;
```

```
  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla
```

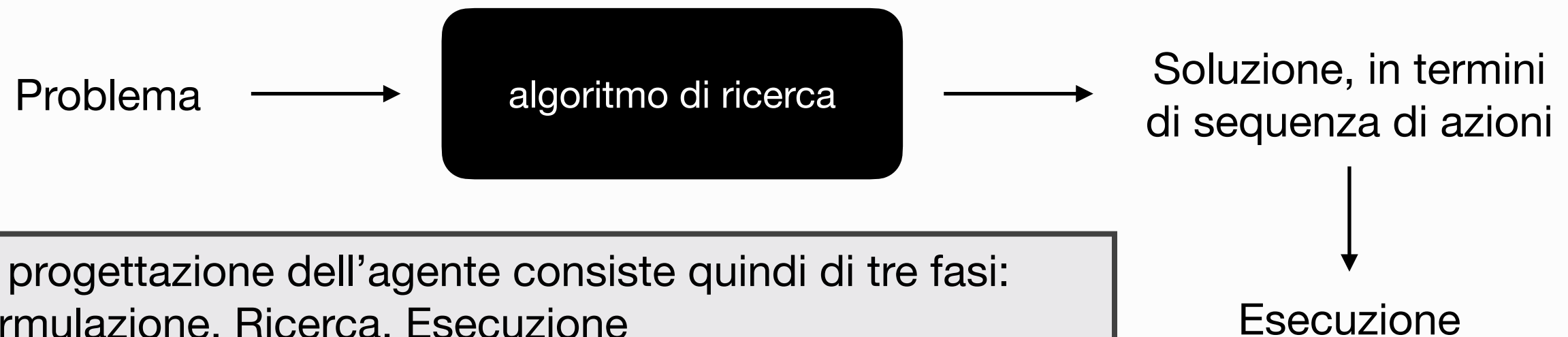
```
  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

/* Aggiornato lo stato, dato lo stato precedente e la percezione attuale, l'agente formulerà un obiettivo e un problema. Dopodiché, avvierà l'algoritmo di ricerca */

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
              stato, una descrizione dello stato corrente del mondo;
              goal, un obiettivo, inizialmente null;
              problema, una formulazione di problema;

  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla

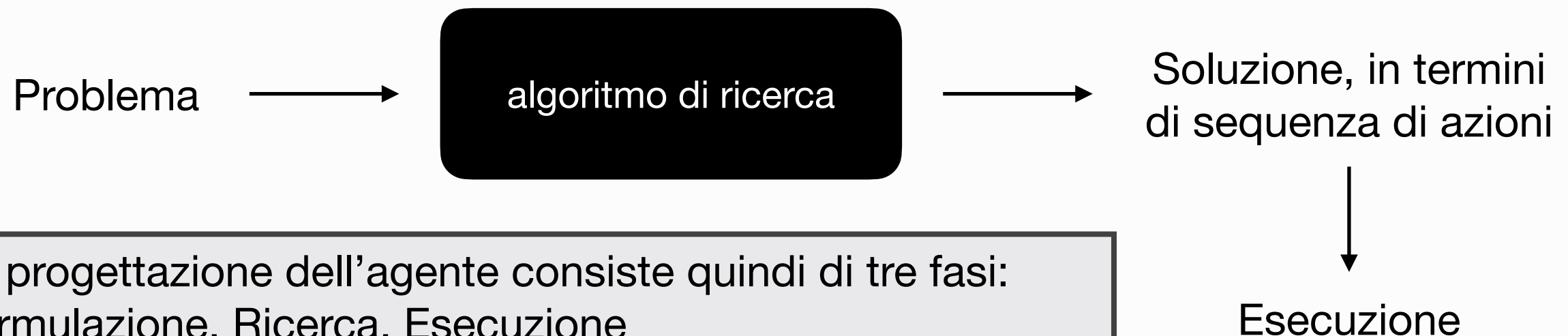
  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

← /* Se l'algoritmo fallisce, l'agente non compirà alcuna azione */

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



La progettazione dell'agente consiste quindi di tre fasi: Formulazione, Ricerca, Esecuzione

```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
               stato, una descrizione dello stato corrente del mondo;
               goal, un obiettivo, inizialmente null;
               problema, una formulazione di problema;

  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla

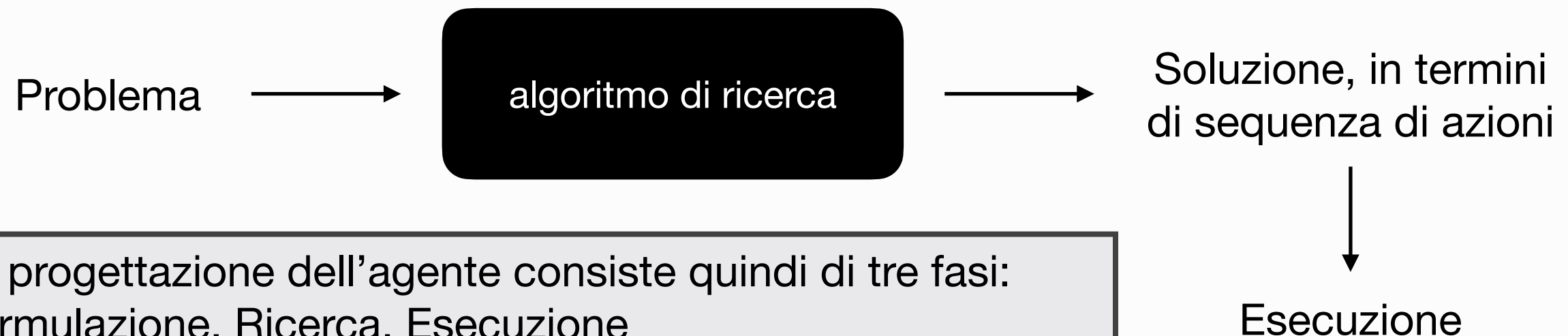
  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

← /* Altrimenti, l'agente inizierà ad eseguire le azioni raccomandate, una alla volta, fino alla fine della sequenza */

Agenti Risolutori di Problemi

Ricerca

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



La progettazione dell'agente consiste quindi di tre fasi: Formulazione, Ricerca, Esecuzione

```
function SEMPLICE-AGENTE-RISOLUTORE-PROBLEMI(percezione)
  persistent: seq, una sequenza di azioni, inizialmente vuota;
               stato, una descrizione dello stato corrente del mondo;
               goal, un obiettivo, inizialmente null;
               problema, una formulazione di problema;

  stato <- AGGIORNA-STATO (stato, percezione)
  if seq è vuota then
    goal <- FORMULA-OBIETTIVO(stato)
    problema <- FORMULA-PROBLEMA(stato, goal)
    seq <- RICERCA(problema)
    if seq = fallimento then return un'azione nulla

  while seq non è vuota
    azione <- PRIMO-ELEMENTO(seq)
    seq <- CODA(seq)
```

Nota bene:

Mentre l'agente esegue la sequenza di azioni *ignora le proprie percezioni*, perché le conosce in anticipo

Un agente di questo tipo, che porta avanti le proprie azioni ad "occhi chiusi", *deve essere certo di quello che accade*.

Nella teoria del controllo, si parla di **sistema a ciclo aperto**, perché ignorando le percezioni si rompe il ciclo tra agente e ambiente.

Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni

Un **problema** può essere definito formalmente da cinque componenti:

- **Stato iniziale.** Rappresenta il punto di partenza dell'agente. Nell'esempio precedente, lo stato iniziale potrebbe essere descritto come *in(Arad)*.
- **Azioni.** L'insieme *Azioni* include la descrizione delle possibili operazioni attuabili dall'agente. Più formalmente, dato uno stato *s*, *Azioni(s)* restituisce l'insieme di azioni che possono essere eseguite in *s*, anche dette azioni **applicabili** in *s*. Nell'esempio precedente, le azioni possibile sono $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.
- **Modello di transizione.** Descrive il risultato di ogni azione attuabile dall'agente in *s*. E' specificato da una funzione *Risultato(s, a)* che restituisce lo stato risultante dall'esecuzione dell'azione *a* nello stato *s*. Utilizziamo anche il termine **successore** per indicare qualsiasi stato raggiungibile da uno stato mediante una singola azione. Ad esempio, potremmo avere che:
 $Risultato(in(Arad), Go(Zerind)) = In(Zerid)$
- **Test obiettivo.** Questo determina se un particolare stato è uno stato obiettivo. In alcuni casi, esiste un insieme esplicito di possibili stati obiettivo: in questo caso il test si limiterà a verificare se uno stato *s* appartiene all'insieme. Nell'esempio, il test obiettivo sarà dato dal singoletto $\{in(Bucarest)\}$.
- **Costo di cammino.** Funzione che determina il costo numerico a ogni cammino. Nell'esempio, il costo potrebbe essere rappresentato dai chilometri da percorrere per arrivare a Bucarest.

Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi

Un **problema** può essere definito formalmente da cinque componenti:

- **Stato iniziale.** Rappresenta il punto di partenza dell'agente. Nell'esempio precedente, lo stato iniziale potrebbe essere descritto come *in(Arad)*.
- **Azioni.** L'insieme *Azioni* include la descrizione delle possibili operazioni attuabili dall'agente. Più formalmente, dato uno stato *s*, *Azioni(s)* restituisce l'insieme di azioni che possono essere eseguite in *s*, anche dette azioni **applicabili** in *s*. Nell'esempio precedente, le azioni possibile sono $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.
- **Modello di transizione.** Descrive il risultato di ogni azione attuabile dall'agente in *s*. E' specificato da una funzione *Risultato(s, a)* che restituisce lo stato risultante dall'esecuzione dell'azione *a* nello stato *s*. Utilizziamo anche il termine **successore** per indicare qualsiasi stato raggiungibile da uno stato mediante una singola azione. Ad esempio, potremmo avere che:
 $Risultato(in(Arad), Go(Zerind)) = In(Zerid)$
- **Test obiettivo.** Questo determina se un particolare stato è uno stato obiettivo. In alcuni casi, esiste un insieme esplicito di possibili stati obiettivo: in questo caso il test si limiterà a verificare se uno stato *s* appartiene all'insieme. Nell'esempio, il test obiettivo sarà dato dal singoletto $\{in(Bucarest)\}$.
- **Costo di cammino.** Funzione che determina il costo numerico a ogni cammino. Nell'esempio, il costo potrebbe essere rappresentato dai chilometri da percorrere per arrivare a Bucarest.

Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi

Un **problema** può essere definito formalmente da cinque componenti:

- **Stato iniziale.** Rappresenta il punto di partenza dell'agente. Nell'esempio precedente, lo stato iniziale potrebbe essere descritto come *in(Arad)*.
- **Azioni.** L'insieme *Azioni* include la descrizione delle possibili operazioni attuabili dall'agente. Più formalmente, dato uno stato *s*, *Azioni(s)* restituisce l'insieme di azioni che possono essere eseguite in *s*, anche dette azioni **applicabili** in *s*. Nell'esempio precedente, le azioni possibile sono $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.
- **Modello di transizione.** Descrive il risultato di ogni azione attuabile dall'agente in *s*. E' specificato da una funzione *Risultato(s, a)* che restituisce lo stato risultante dall'esecuzione dell'azione *a* nello stato *s*. Utilizziamo anche il termine **successore** per indicare qualsiasi stato raggiungibile da uno stato mediante una singola azione. Ad esempio, potremmo avere che:
 $Risultato(in(Arad), Go(Zerind)) = In(Zerind)$

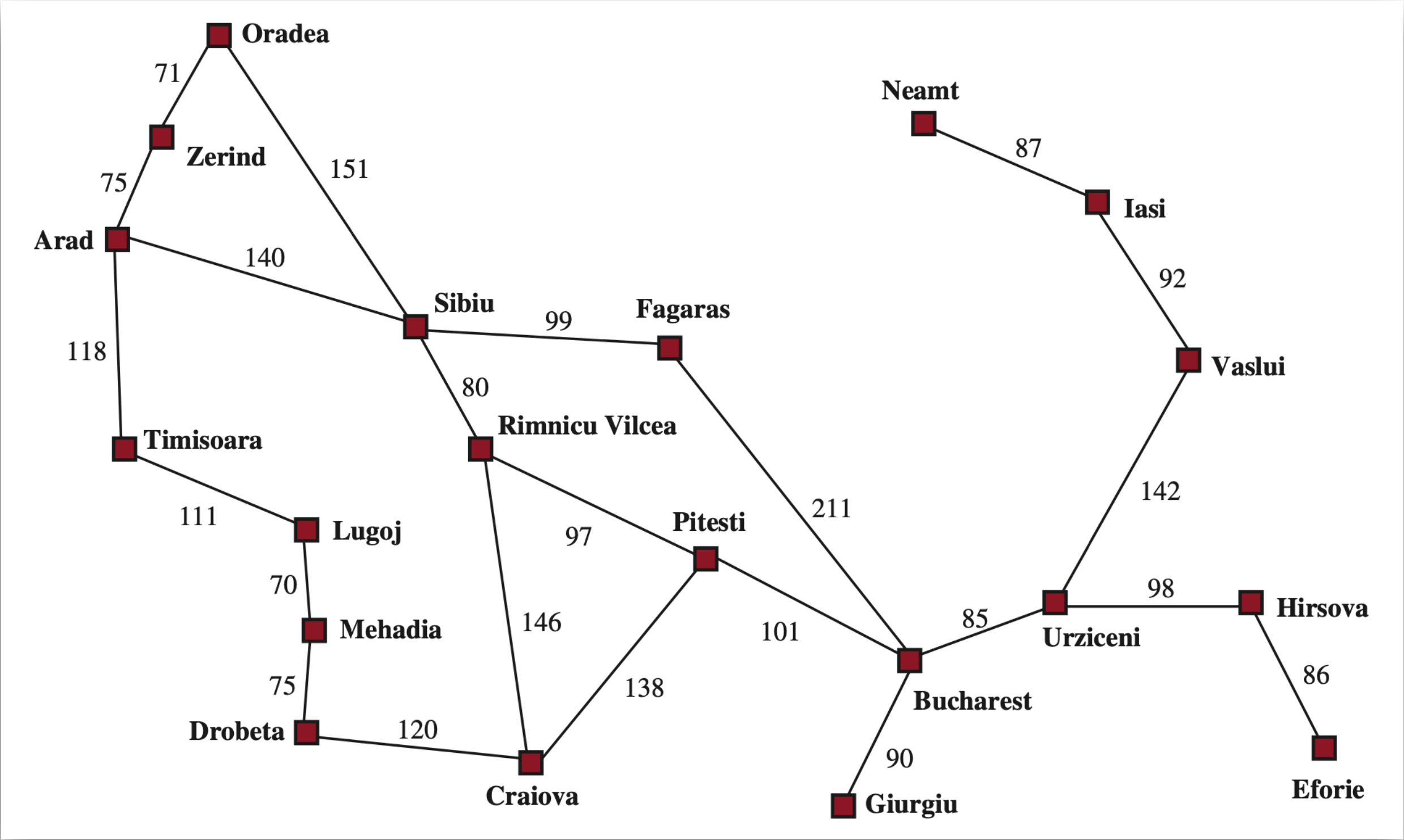
Stato iniziale, azioni, e modello di transizione definiscono lo **stato degli spazi** del problema, l'insieme di tutti gli stati raggiungibili da quello iniziale mediante qualsiasi sequenza di azioni.

Questo può essere rappresentato sotto forma di **grafo**, in cui i nodi rappresentano gli stati e gli archi (pesati) rappresentano l'azione che porta da un nodo ad un altro, etichettata con il relativo **costo di passo** $c(s, a, s')$, ovvero il costo dell'azione *a* che porta dallo stato *s* a quello *s'*.

arrivare a Bucarest.

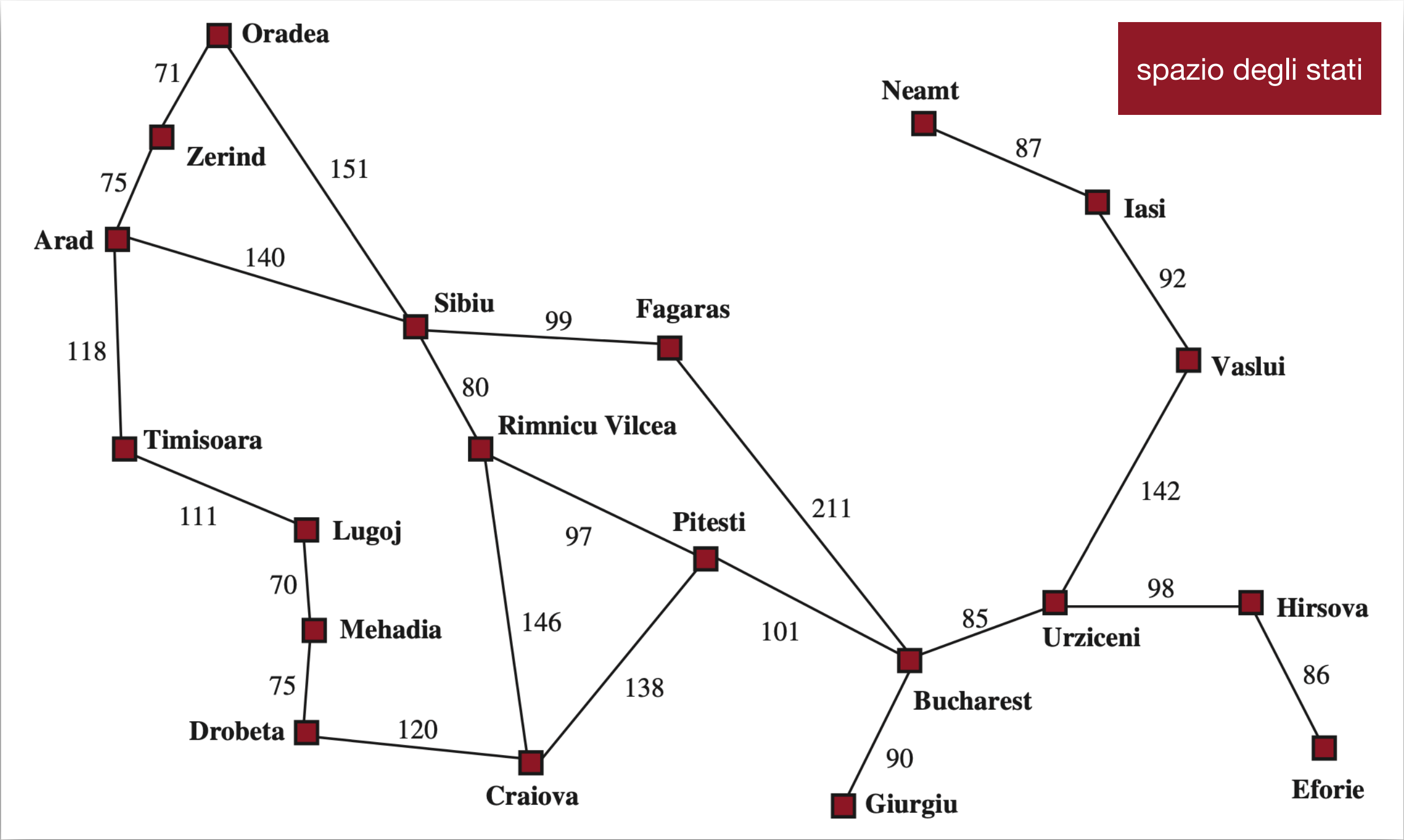
Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi



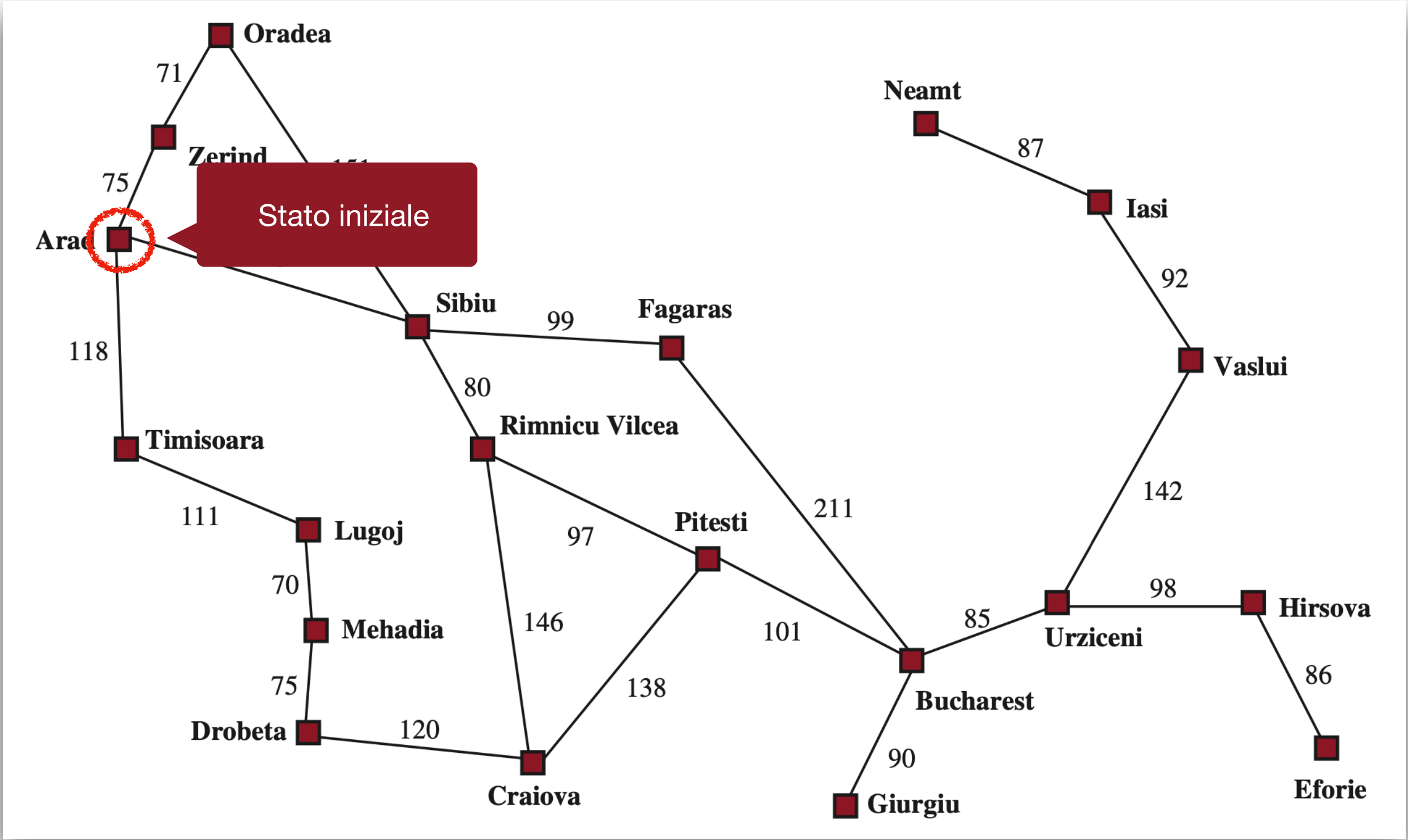
Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi



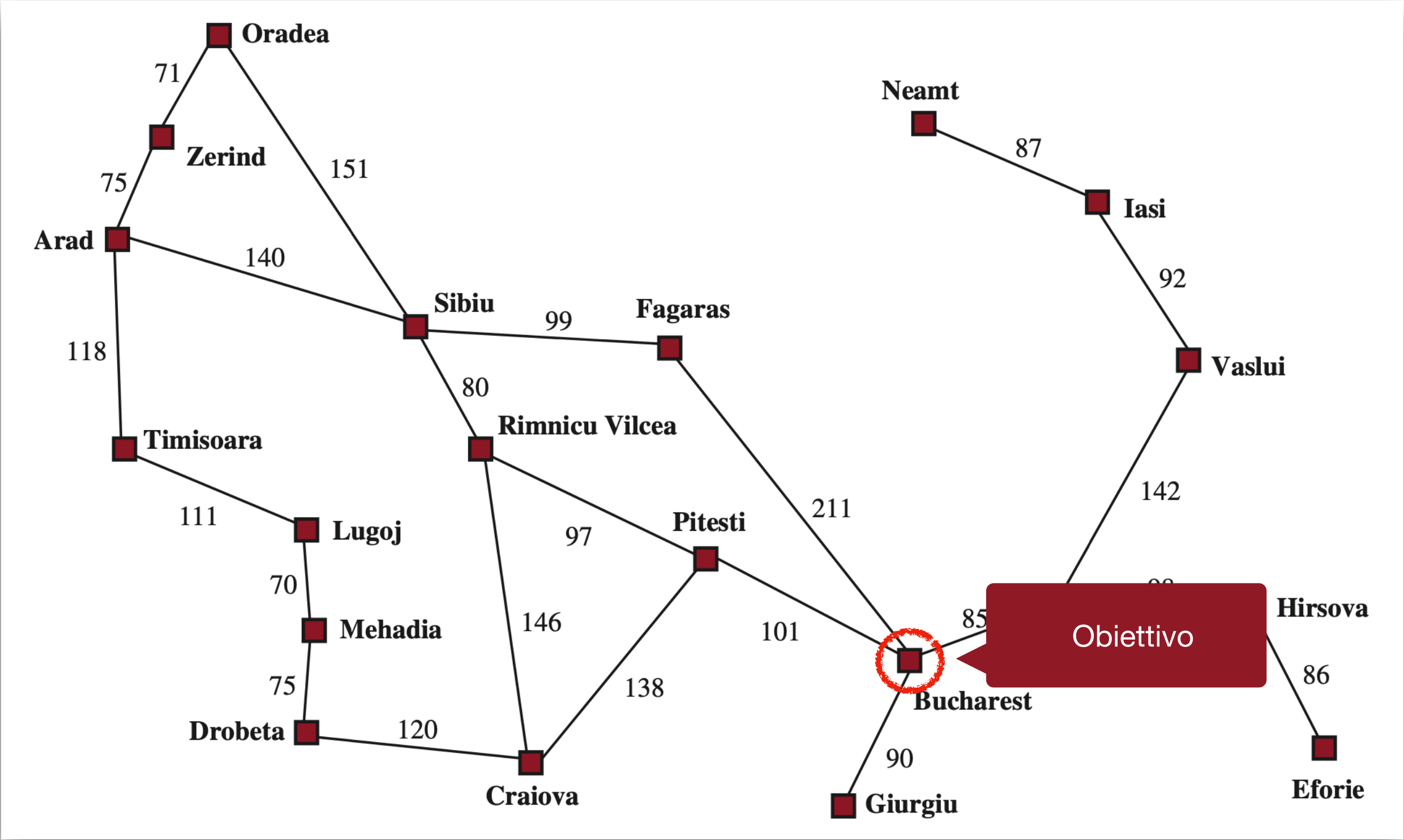
Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi



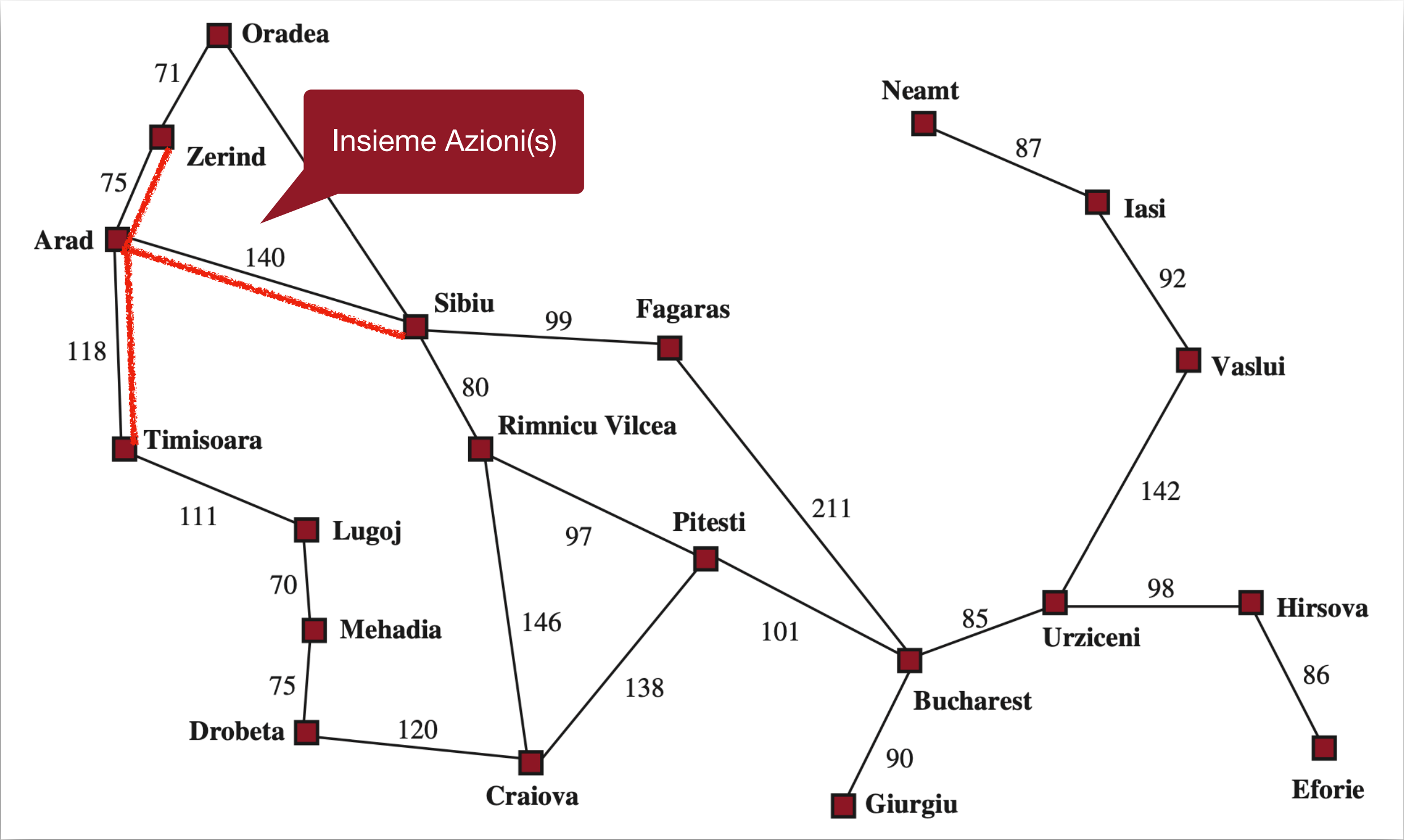
Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi



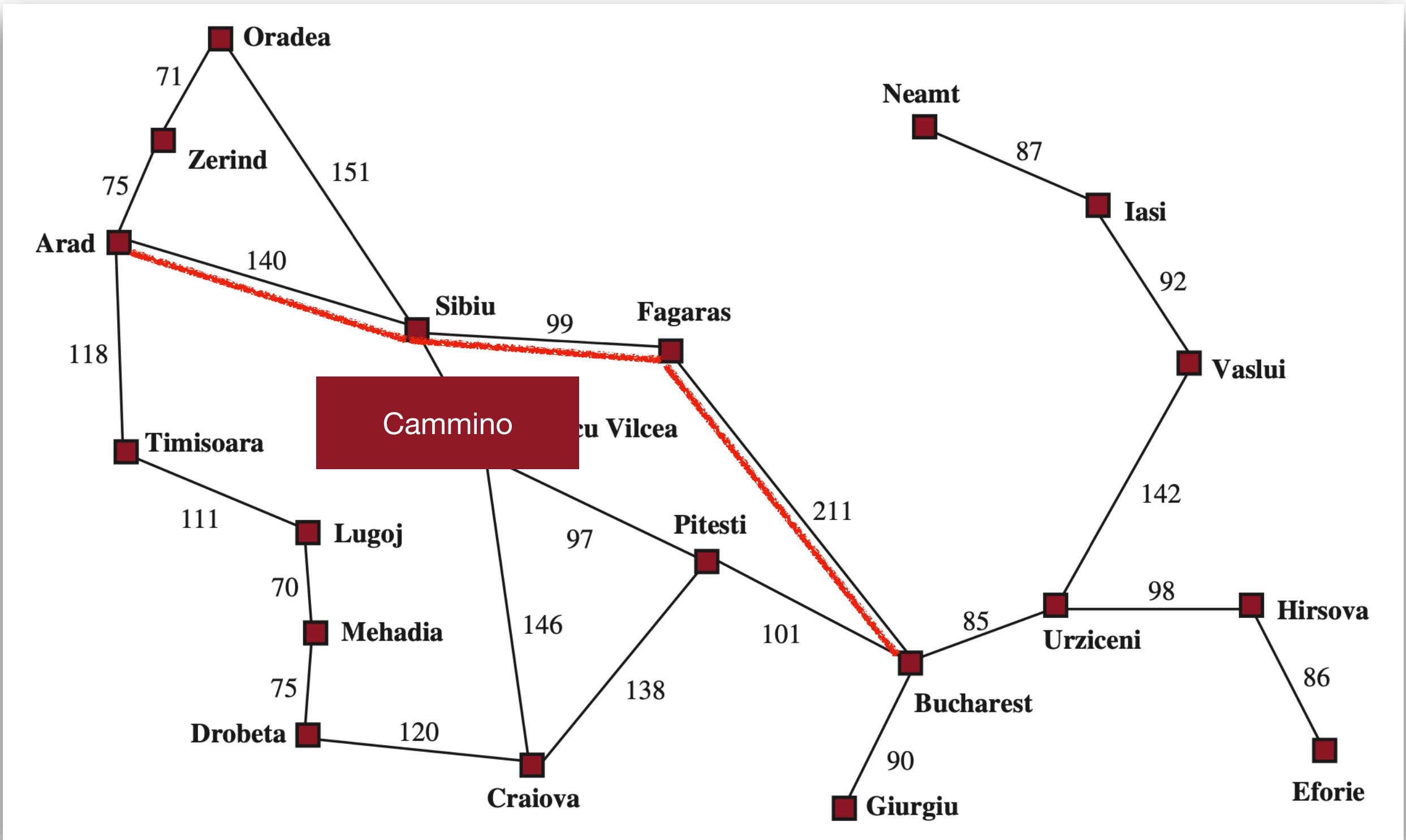
Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi



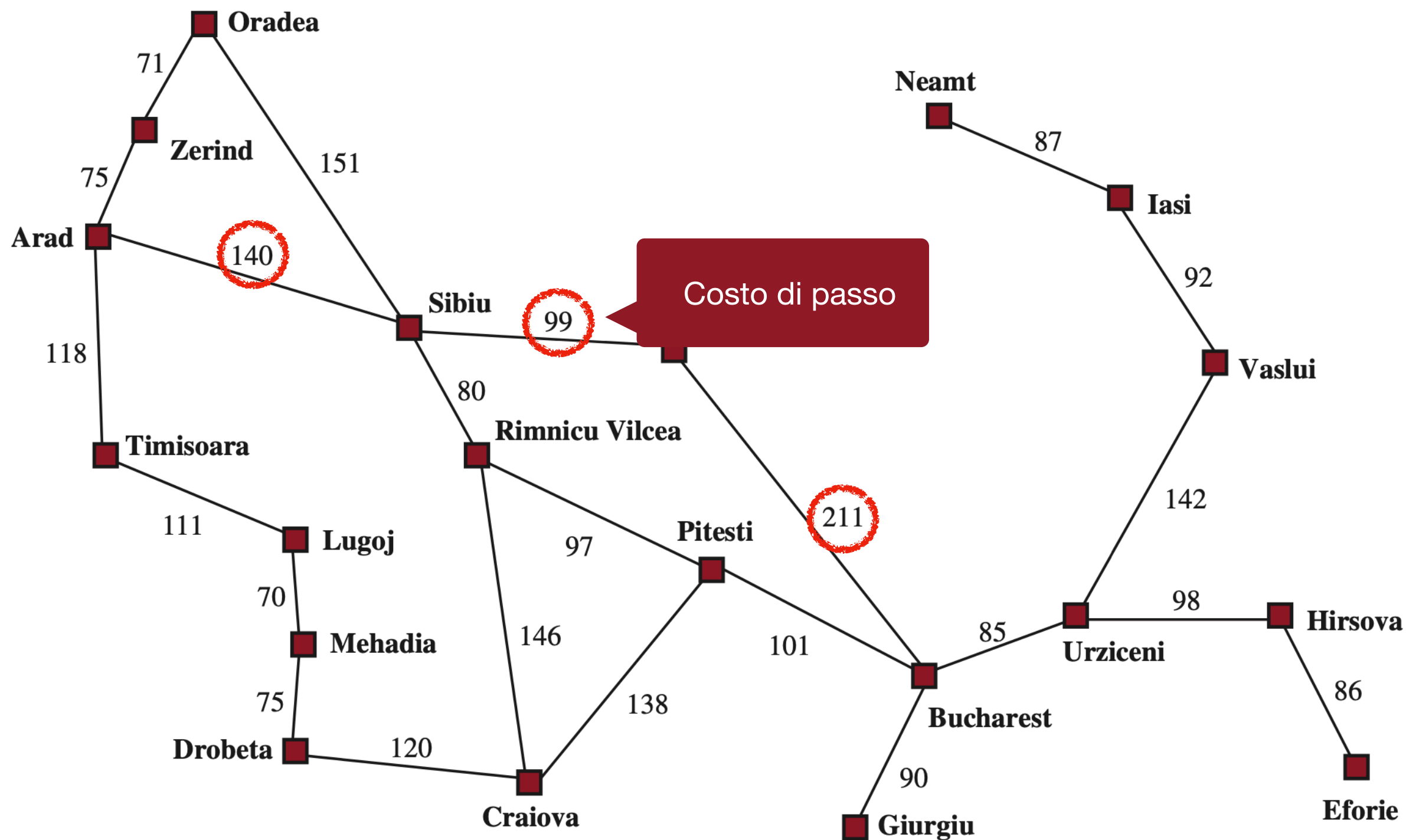
Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi



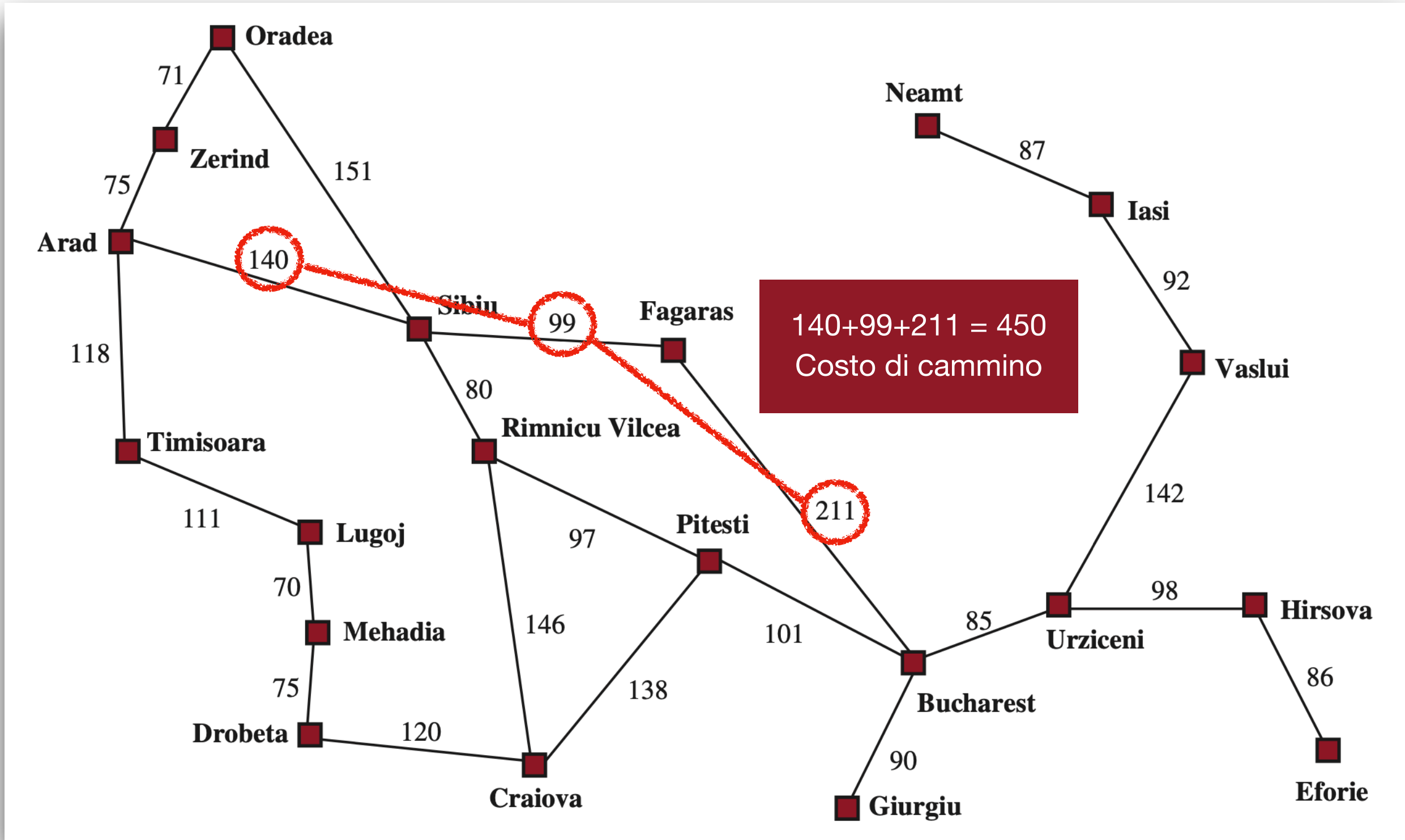
Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni - Stato degli Spazi



Agenti Risolutori di Problemi

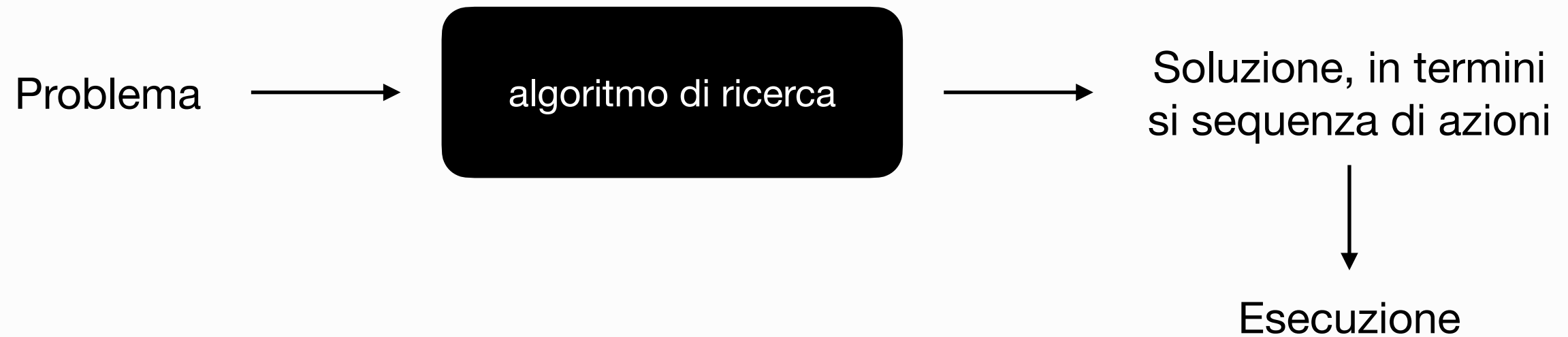
Problemi ben definiti e soluzioni - Stato degli Spazi



Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni

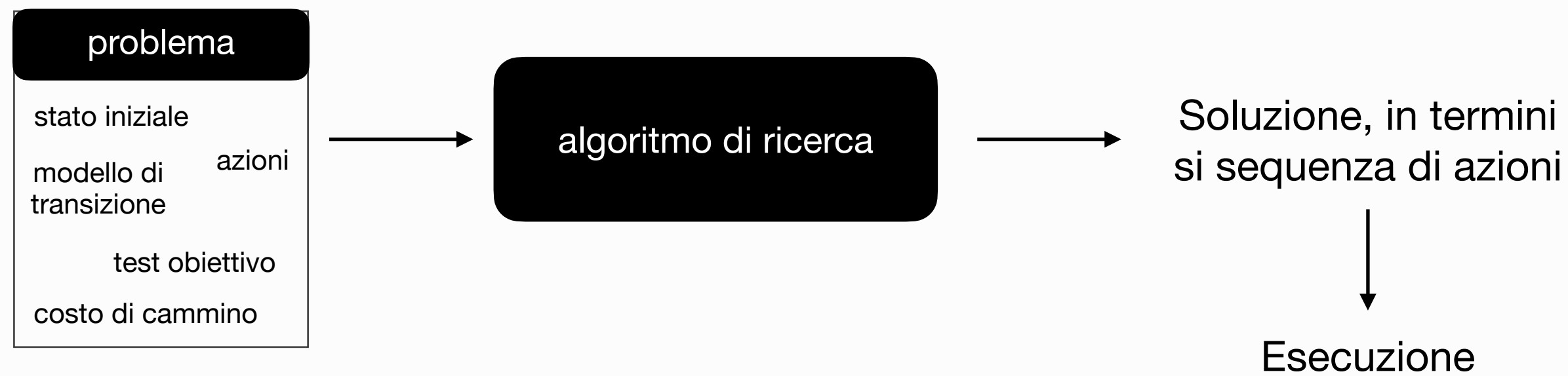
Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni

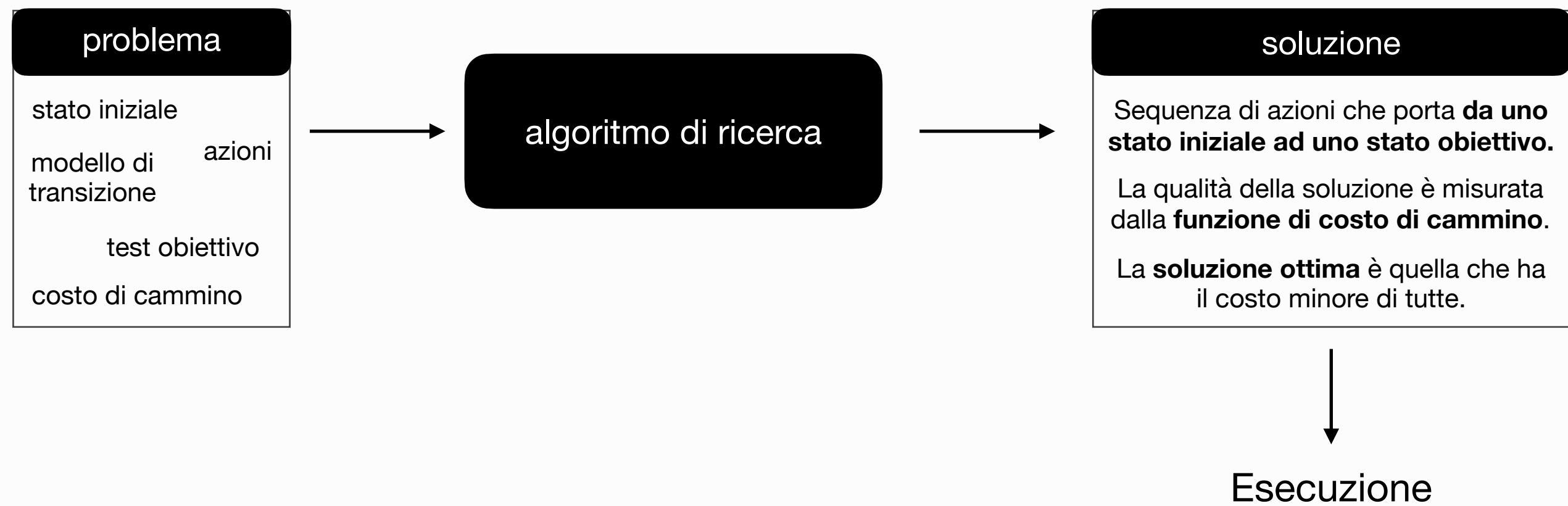
Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni

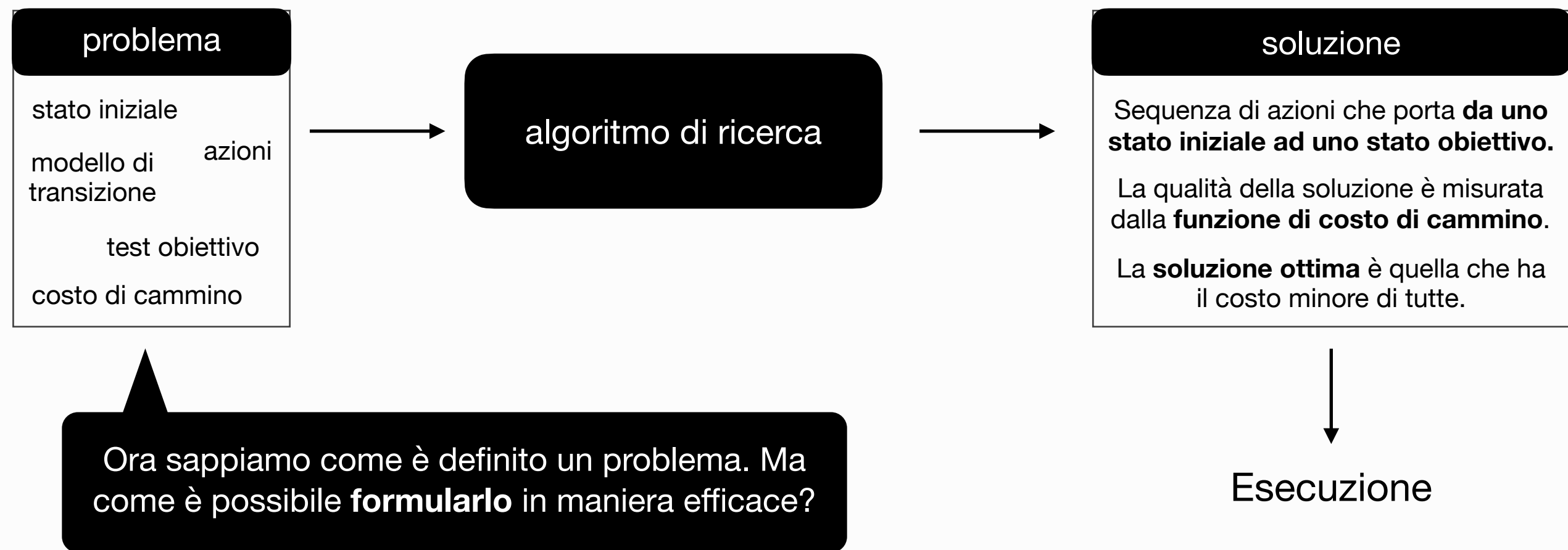
Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



Agenti Risolutori di Problemi

Problemi ben definiti e soluzioni

Il processo che cerca una sequenza di azioni che raggiunge l'obiettivo è detto **ricerca**.



Agenti Risolutori di Problemi

Formulazione di problemi

Nell'esempio precedente, abbiamo proposto la seguente formulazione del problema:

- **Stato iniziale.** $in(Arad)$.
- **Azioni.** $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.
- **Modello di transizione.** $Risultato(in(Arad), Go(Zerind)) = In(Zerind)$
- **Test obiettivo.** $\{in(Bucarest)\}$.
- **Costo di cammino.** Chilometri da percorrere per arrivare a Bucarest.

Sebbene sembri ragionevole, questa formulazione rappresenta una *descrizione matematica astratta*, ovvero **un modello**.

Basti confrontare la descrizione di stato scelta, $in(Arad)$, con un vero viaggio in macchina. In un contesto del genere, dovremo passare necessariamente per una serie di altri paesi e città. Lo stato del mondo include varie altre particolarità che influenzano l'agente, come i compagni di viaggio, la presenza di poliziotti nelle vicinanze, le condizioni del tempo e della strada, ecc.

Queste considerazioni sono state omesse poiché sono irrilevanti al fine ultimo di raggiungere l'obiettivo Bucarest. Il processo di rimozione dei dettagli da una rappresentazione è detta **astrazione**.

Agenti Risolutori di Problemi

Formulazione di problemi

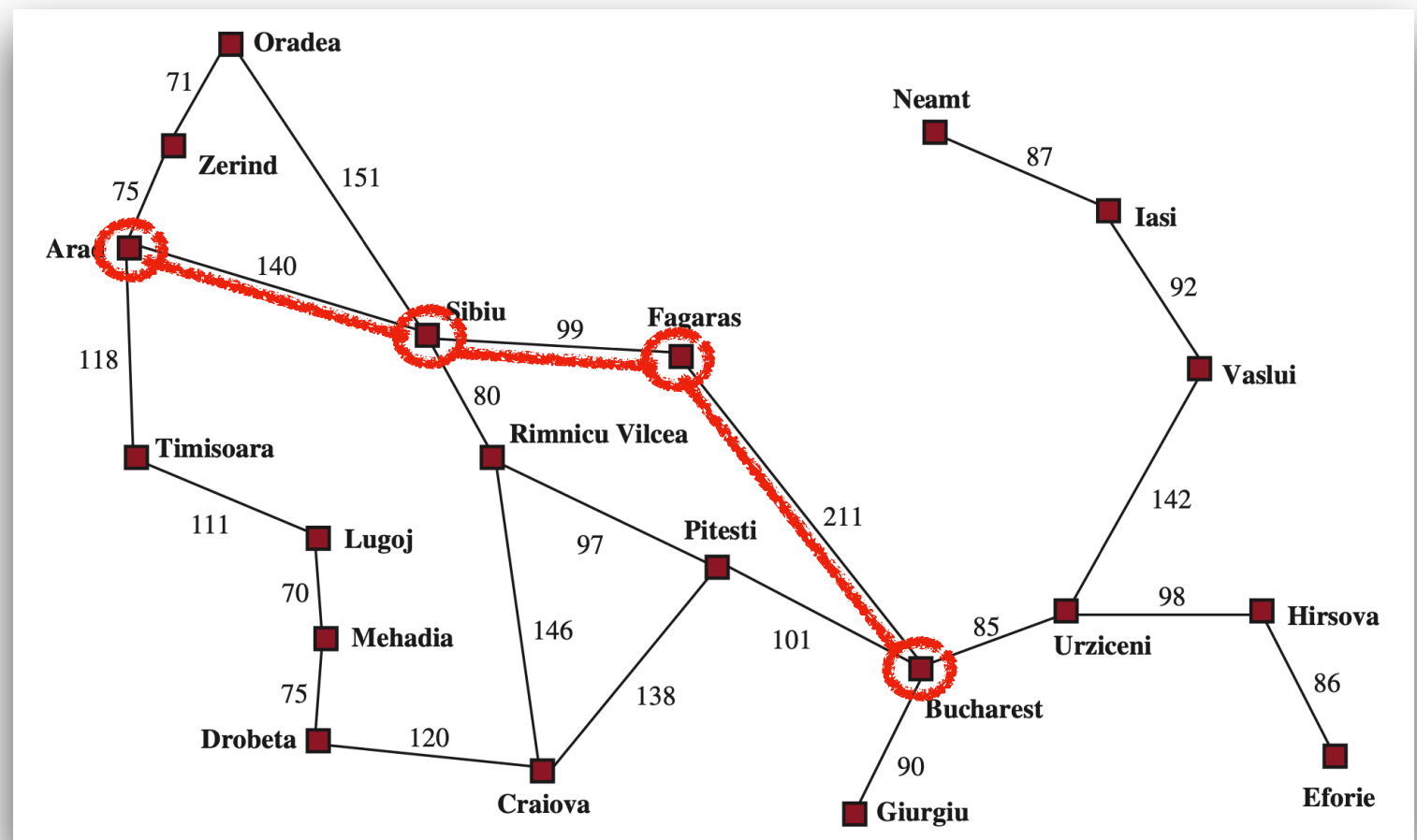
Oltre ad astrarre le descrizioni di stato, dobbiamo astrarre anche le azioni - un'azione di guida è influenzata e può avere molti effetti (ad esempio, la benzina viene consumata).

E' possibile essere più precisi nella definizione di un livello di astrazione *appropriato*?

Possiamo pensare ad uno stato non come un elemento individuale, ma come un **insieme di stati dettagliati**; allo stesso modo, un'azione è in realtà un **insieme di azioni dettagliate**.

In altri termini, con questa formulazione del problema il viaggio a Bucarest sarebbe scomponibile in sotto-problemi:

1. Da Arad a Sibiu;
2. Da Sibiu a Faragas;
3. Da Faragas a Bucarest.



Agenti Risolutori di Problemi

Formulazione di problemi

Oltre ad astrarre le descrizioni di stato, dobbiamo astrarre anche le azioni - un'azione di guida è influenzata e può avere molti effetti (ad esempio, la benzina viene consumata).

E' possibile essere più precisi nella definizione di un livello di astrazione *appropriato*?

Possiamo pensare ad uno stato non come un elemento individuale, ma come un **insieme di stati dettagliati**; allo stesso modo, un'azione è in realtà un **insieme di azioni dettagliate**.

Un'astrazione si definisce **valida** se possiamo espandere ogni soluzione astratta in una soluzione nel mondo più dettagliata.

Un'astrazione si definisce **utile** se eseguire ogni azione nella soluzione è più facile che nel problema originale.

La scelta di una buona astrazione (o granularità dell'astrazione) prevede, quindi, che si rimuova quanto più dettaglio possibile mantenendo la validità e assicurandosi che le azioni astratte siano più facile da eseguire.

Senza la capacità di astrazione, *gli agenti intelligenti sarebbero completamente soverchiati dalla complessità del mondo reale!*

Agenti Risolutori di Problemi

Formulazione di problemi tramite esempi - Il Problema delle 8 Regine

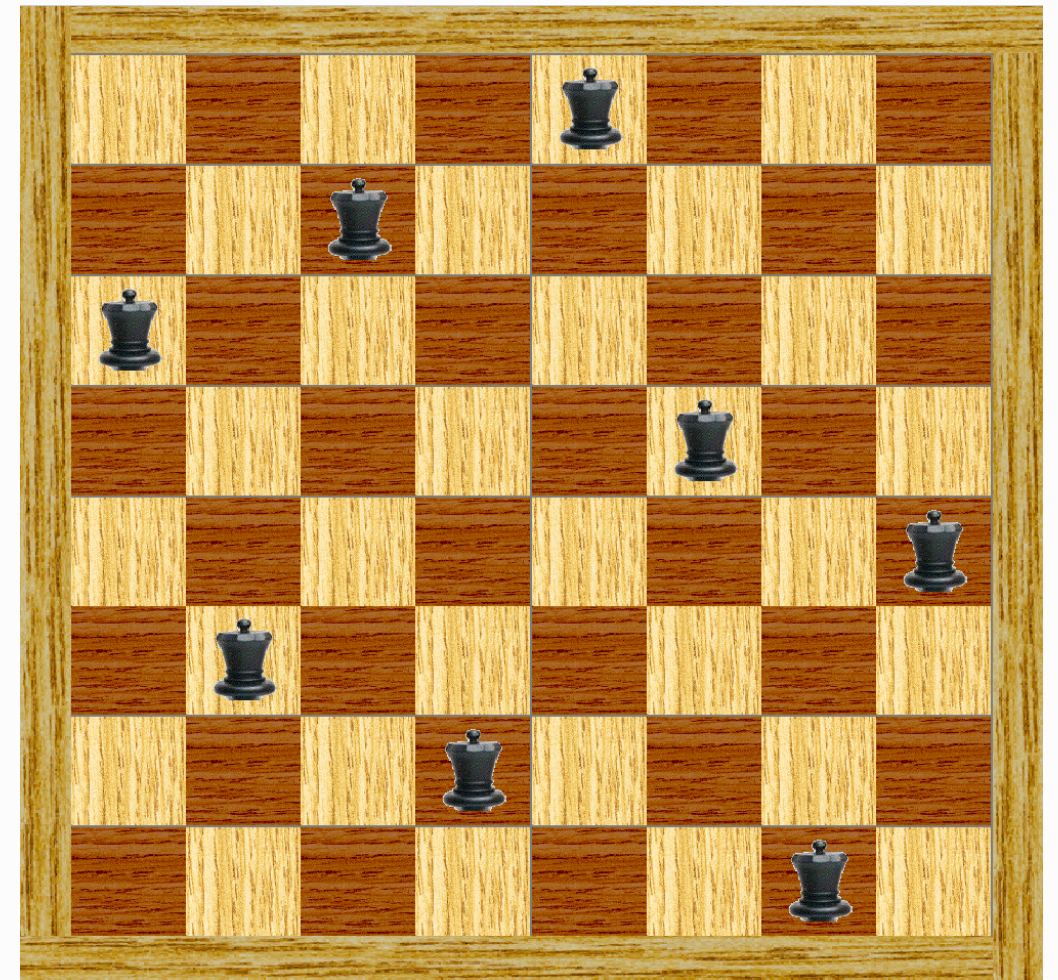
Una volta stabilite le basi di un agente risolutore di problemi così come di un algoritmo di ricerca, consideriamo alcuni casi tipici e/o più pratici di formulazione di problemi.

Scopo. Piazzare 8 regine su una scacchiera in modo tale che nessuna possa attaccarne un'altra. Negli scacchi, una regina attacca i pezzi che si trovano sulla sua stessa colonna, riga o diagonale.

Soluzione. Una delle possibili soluzioni è quella mostrata in figura.

Nell'ambito del corso, vedremo come risolvere il problema utilizzando due particolari tipologie di algoritmi di ricerca, ovvero gli *algoritmi genetici* e il *backtracking*. Per il momento, il nostro obiettivo è quello di definire una corretta formulazione del problema.

In particolare, possiamo parlare di due tipologie di formulazione, che prendono il nome di **formulazione incrementale** e **formulazione a stato completo**.



Agenti Risolutori di Problemi

Formulazione di problemi tramite esempi - Il Problema delle 8 Regine

La formulazione incrementale utilizza operatori che *estendono progressivamente* la descrizione di stato, cominciando dallo stato vuoto.

Stati. Ogni piazzamento sulla scacchiera di un numero da 0 a 8 regine è uno stato.

Stato iniziale. Scacchiera vuota.

Azioni. Aggiungere una regina in una casella vuota.

Modello di transizione. Restituisce la scacchiera con una regina aggiunta nella casella specificata.

Test obiettivo. Sulla scacchiera ci sono 8 regine e nessuna è attaccata.

In questa formulazione abbiamo circa $1,8 \cdot 10^{14}$ possibili sequenze da investigare. Da notare che nessun vincolo è stato specificato nella formulazione.

Stati. Tutte le possibili configurazioni di regine, una per colonna a partire da sinistra, tali che nessuna regina ne attacchi un'altra.

Azioni. Aggiungere una regina in una qualsiasi casella della colonna vuota più a sinistra, in modo tale che essa non sia attaccata da nessun'altra regina.

Questa formulazione riduce lo spazio degli stati a 2.057, rendendo il problema più semplice da risolvere. Ma attenzione, se parliamo del problema generico delle N regine con $N=100$, la riduzione porterebbe a 10^{52} stati, il che renderebbe il problema intrattabile.

Agenti Risolutori di Problemi

Formulazione di problemi tramite esempi - Il Problema delle 8 Regine

La formulazione a stati completi prevede che le otto regine siano già sulla scacchiera, per poi spostarle in maniera iterativa.

Stati. Ogni piazzamento sulla scacchiera di un numero da 0 a 8 regine è uno stato.

Stato iniziale. Scacchiera con una regina piazzata per colonna.

Azioni. Sposta una regina nella colonna, se minacciata.

Modello di transizione. Restituisce la scacchiera con una regina la cui posizione è modificata.

Test obiettivo. Sulla scacchiera ci sono 8 regine e nessuna è attaccata.

In questa formulazione abbiamo circa $1,6 \cdot 10^7$ possibili sequenze da investigare, notevolmente di meno rispetto all'equivalente formulazione incrementale senza vincoli.

Agenti Risolutori di Problemi

Formulazione di problemi tramite esempi - Il Problema del Commesso Viaggiatore

Il problema del commesso viaggiatore è un altro tipico esempio di utilizzo di algoritmi di ricerca. A differenza del caso precedente, parliamo di problemi di ottimizzazione *reali*.

Scopo. Un commesso deve consegnare dei pacchi e, pertanto, dovrà visitare ogni città esattamente una volta: l'obiettivo è trovare il percorso più breve. Il problema è spesso implementato in molte applicazioni logistiche.

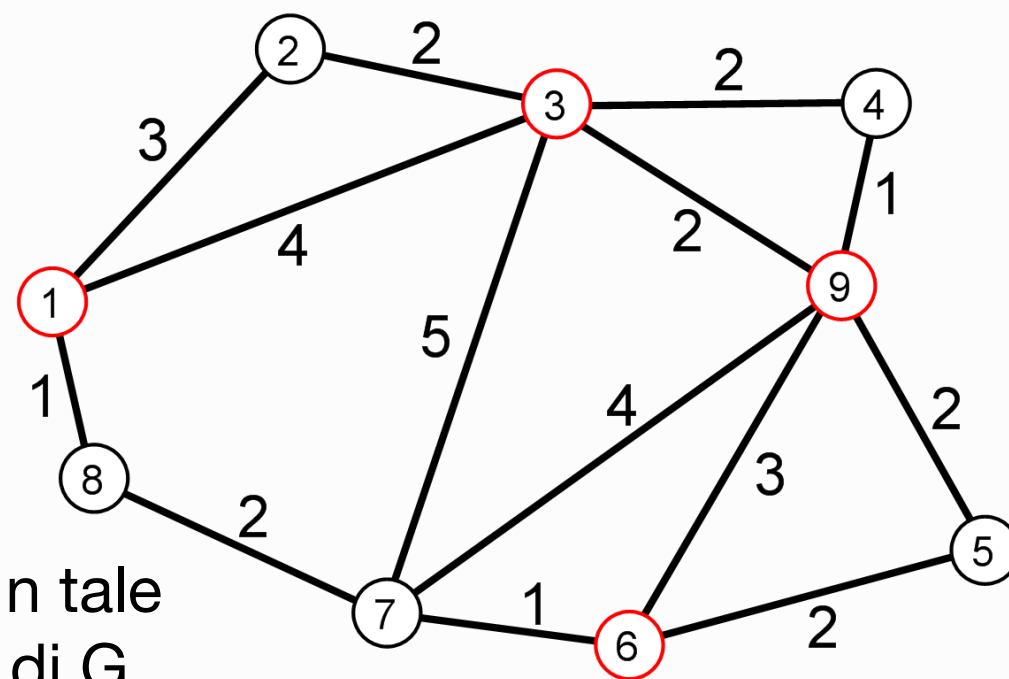
Un ciclo che passa su tutti i nodi del grafo una e una sola volta è detto *ciclo Hamiltoniano*. Determinare se un grafo contiene un ciclo di questo tipo rappresenta un problema NP-Completo (non è risolvibile in tempo polinomiale).

Grafo $G(N, A)$ completo - ovvero un grafo che ha un arco tra ogni coppia di vertici.

c_{ij} = costi sugli archi.

La risposta al problema è 'Sì' se esiste un ciclo hamiltoniano nel grafo G , 'No' altrimenti.

Formalmente, una permutazione π dei nodi $1, 2, \dots, n$ tale che per ogni $i = 1, \dots, n - 1$, $(\pi(i), \pi(i + 1))$ è un arco di G .



Agenti Risolutori di Problemi

Formulazione di problemi tramite esempi - Il Problema del Commesso Viaggiatore

Il problema del commesso viaggiatore è un altro tipico esempio di utilizzo di algoritmi di ricerca. A differenza del caso precedente, parliamo di problemi di ottimizzazione *reali*.

Stati. Posizione corrente + *insieme di città già visitate*.

Stato iniziale. Il commesso si trova nella prima città.

Azioni. Spostamenti tra i nodi (città).

Modello di transizione. Restituisce un grafo modificato in cui il commesso si è spostato da una città ad un'altra.

Test obiettivo. Il commesso arriva all'ultima città passando per tutte le città.

Agenti Risolutori di Problemi

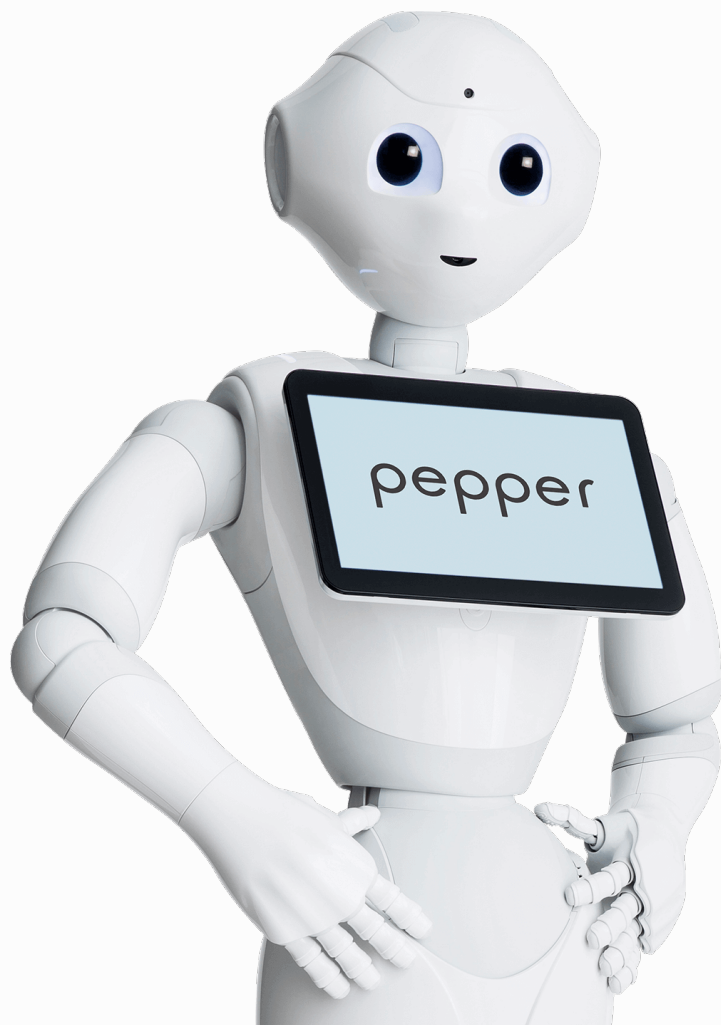
Formulazione di problemi tramite esempi - Altri problemi risolvibili con la ricerca

La navigazione dei robot, ad esempio la navigazione di un'aspirapolvere smart, è una generalizzazione del problema di ricerca di itinerario descritto precedentemente.

Invece di seguire un insieme discreto di stati, un robot si può muovere in uno spazio continuo e avere, almeno in via di principio, *un insieme infinito di possibili stati e azioni*.

Quando il robot è dotato di braccia o gambe che devono essere controllate, lo spazio di ricerca diventa *multi-dimensionale*.

Se possibile, nell'ambito del corso vedremo come poter sviluppare un algoritmo di ricerca per la navigazione di Pepper.





UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Laurea triennale in Informatica

Fondamenti di Intelligenza Artificiale

Lezione 3 - Formulazione di problemi di ricerca



Agenti Risolutori di Problemi

Esercizi consigliati

Fornire una formulazione completa per ciascuno dei seguenti problemi:

1. Una scimmia alta 90 cm si trova in una stanza con delle banane appese al soffitto, che è alto 3 metri. La scimmia vorrebbe raggiungere le banane. La stanza contiene due casse alte ognuna un metro; le casse si possono muovere, scalare, impilare.
2. Ci sono tre brocche rispettivamente da 12, 8 e 3 litri e un rubinetto. Potete riempire le brocche fino all'orlo e svuotarle una dentro l'altra o nel lavandino. Dovete ottenere esattamente un litro d'acqua.
3. Colorare una mappa bidimensionale usando solo quattro colori, in modo tale che non ci siano regioni adiacenti con lo stesso colore.